

CMPT 295

Textbook
Chapter 3

Unit - Machine-Level Programming

Lecture 9 – Introduction - Compilation process:

C -> assembly code -> machine level code

Last Lecture

- Most real numbers **R** cannot be **exactly** encoded using **IEEE 754 standard floating point representation**, they are **approximated** when encoded
 - This happens because most **R** need more than 23 bits to encode the binary version of their fractional part (**frac**)
 - Therefore their **frac** is subject to **rounding**, hence **approximation**
- 1. **Round up**: add 1 to bit at **rounding position** and discard bits on right of **rounding position**
- 2. **Round down**: discard bits on right of **rounding position**
- 3. **Round to even number**: either **Round up** or **Round down**
- **Denormalized** values = $\pm 0.\text{frac} \times 2^{-126}$ (single precision)
 - Condition: **exp** = 00000000
 - **Case 1**: **frac** = 000...0 $\rightarrow$ +0 and -0
 - **Case 2**: **frac** $\neq$ 000...0 $\rightarrow$ #s closest to 0.0
 - All numbers equidistant because all have same 2^E and their **M** increments linearly
- **Special values** (single precision)
 - Condition: **exp** = 11111111
 - **Case 1**: **frac** = 000...0 $\rightarrow$ $+\infty$ and $-\infty$ (infinity)
 - **Case 2**: **frac** $\neq$ 000...0 $\rightarrow$ NaN (Not-a-Number)

Appropriate
Equations:

$$E = 1 - \text{bias}$$

$$\text{bias} = 2^{k-1} - 1$$

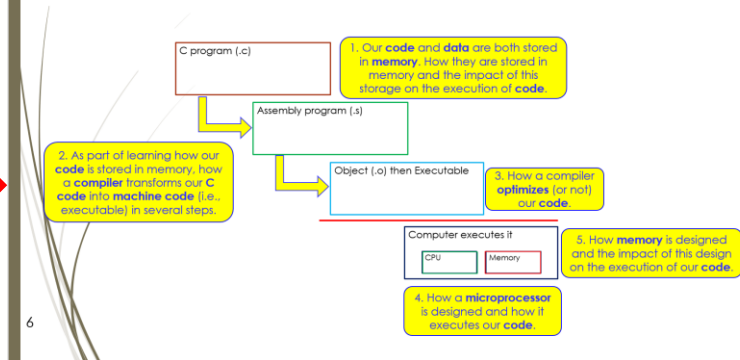
$$M = \text{frac}$$

➤ Smallest Denormalized value:	0 00000000 000000000000000000000001
$V = (-1)^s M 2^E = 0.000000000000000000000001 \times 2^{-126} = \sim 1.4 \times 10^{-45}$	
➤ Largest Denormalized value:	0 00000000 111111111111111111111111
$V = (-1)^s M 2^E = 0.111111111111111111111111 \times 2^{-126} = \sim 1.18 \times 10^{-38}$	

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

In CMPT 295, we shall learn ...



In Lecture 3, we asked ...

What could these 32 bits represent?

- What kind of information could they encode?

0 1 1 0 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 1 0 1 0 0 0 1 1 1 0 0 1 1₂

Answer:

- Aside from characters, integers or floating point numbers, etc...
- **Review:** We saw that all modern computers, designed based on the **von Neumann architecture**, store their programs in memory
 - Data and instructions of our **C** program are in main memory together (but in different locations)
- So, the above 32 bits could also represent code
 - For example:
 - Assembly code: `sub $0x18, %rsp`
 - Machine code: `48 83 ec 18`

C program in memory?

We have spent a few lectures looking at how our **data** can be represented in memory as a series of 0's and 1's

Now, we shall look at ...

1. How our **C** program/code can be represented as a series of 0's and 1's (i.e., as machine code)
2. Then, how our **C** program/code, once it is represented as a series of 0's and 1's, ends up being stored in memory
3. Then, how our **C** program (once it is represented as a series of 0's and 1's and it is stored in memory) ends up being executed by the microprocessor (aka the CPU)

Demo – C program: sum_store.c

1. How our C program/code can be represented as a series of 0's and 1's (i.e., as machine code)

Let's answer these questions with a demo

Turning **C** into machine code - **gcc**

The Big Picture

C program (.c) → `sum_store.c`

1. How our **C** program/code can be represented as a series of 0's and 1's (i.e., as machine code)

C Preprocessor `gcc -E sum_store.c` → `sum_store.i` *text*

→ `sum_store.i`

C Compiler `gcc -Og -S sum_store.i` OR `gcc -Og -S sum_store.c`

Assembly program (.s) → `sum_store.s`

Assembler `gcc -g -c sum_store.s` *text*

Object (.o) → `sum_store.o`

Linker `gcc -o ss main.o sum_store.o` *binary* *binary*

Executable → `ss`

Loader `./ss 5 6`

2. Then, how our **C** program/code, once it is represented as a series of 0's and 1's, ends up being stored in memory

optional
Disassembled object file
text & binary

Disassembler
`objdump -d ss`
gdb debugger

Snapshot of compilation process

```
*dest = sum;
```

➤ C code

➤ Store **sum**
in memory designated by pointer **dest**

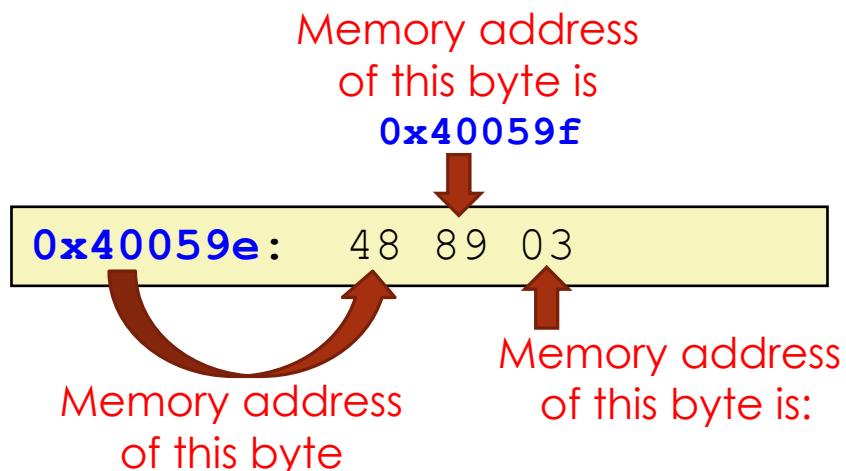
```
movq %rax, (%rbx)
```

➤ Assembly code

➤ Move an 8-byte value to memory
➤ **Quad words** in x86-64 terminology

➤ Operands:

sum:	Register	%rax
dest:	Register	%rbx
*dest:	Memory	M[%rbx]



➤ Machine code (0's and 1's): **48 89 03**

➤ 3-byte instruction

➤ Stored at address **0x40059e**

Fetch-Execute Cycle

PC: program counter

Defⁿ: register containing **address** of instruction of **ss** that is to be executed next

IR: instruction register

Defⁿ: register containing **copy** of instruction of **ss** that is currently executing

3. Then, how our **C** program (once it is represented as a series of 0's and 1's and it is stored in memory) ends up being executed by the microprocessor (aka the CPU)

Explanation: The microprocessor executes the machine code version of our **C** program by executing the following simple loop:

DO FOREVER:

fetch next instruction from memory into CPU and put it into the *instruction register* (**IR**)

update the *program counter* (**PC**)

decode the instruction

execute the instruction

Programming in **C** versus in **x86-64** assembly language

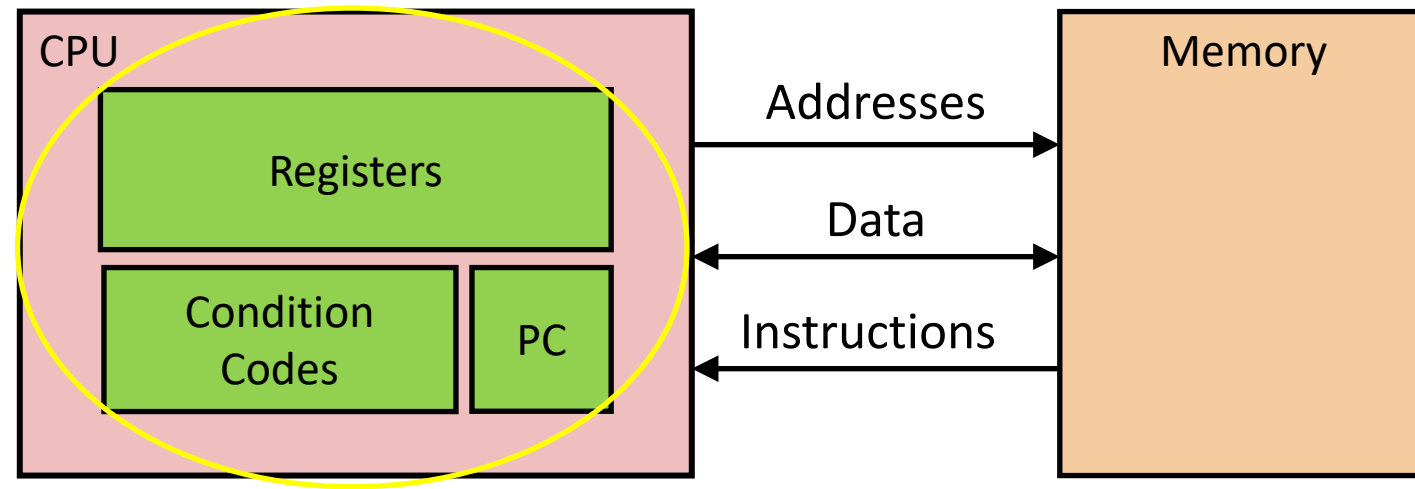
When programming in **C**, we can ...

- Store/retrieve data into/from memory, i.e. variables
- Perform calculations on data
 - e.g., arithmetic, logic, shift
- Transfer control: decide what part of the program to execute next based on some condition
 - e.g., if-else, loop, function call

When programming in **assembly language**, we can do the same things and in addition ...

Programming in x86-64 assembly

- ... we now have access to parts of the microprocessor that are normally hidden from C programmers
- For example, as assembly programmers, we now have access to ...



Hum ... Why are we learning assembly language?

Hum ... Why are we learning assembly language?

Answer:

- To be able to read assembly code and appreciate how a compiler transforms our **C** code into assembly code as well as how it optimizes our **C** code
- To be able to write assembly code and do our own optimizations
- Sometimes what we want our software to do cannot be achieved using **high level programming languages** such as **C**
 - We must then use **low level programming languages** (assembly languages) to achieve it

x86-64 Assembly Language - Data

- **Integral** numbers not stored in variables but in **registers**
 - Different integer sizes: 1, 2, 4 and 8 bytes
 - We indicate the size of our data by the **register** names we use
- **Memory addresses** not stored in pointer variables but in **registers**
 - Size: 8 bytes
 - Treated as integral numbers
- **Floating point numbers** stored in different **registers** than integral values
 - Different floating point number sizes: 4 and 8 bytes
 - We indicate the size of our data by the **register** names we use
- No **compound types** such as arrays or structures

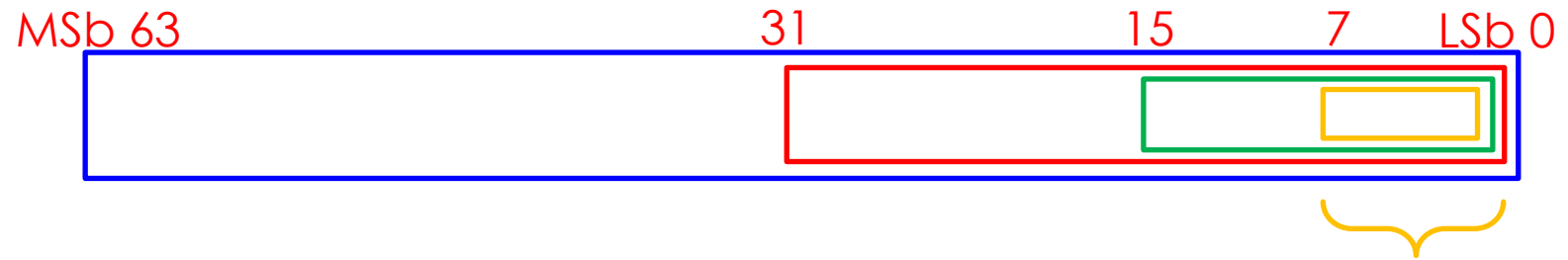
x86-64 Assembly Language – Data

Integer Registers

64-bit (quad)	32-bit (double)	16-bit (word)	8-bit (byte)	
63..0	31..0	15..0		7..0
rax	eax	ax		al
rbx	ebx	bx		bl
rcx	ecx	cx		cl
rdx	edx	dx		dl
rsi	esi	si		sil
rdi	edi	di		dil
rbp	ebp	bp		bpl
rsp	esp	sp		spl
r8	r8d	r8w		r8b
r9	r9d	r9w		r9b
r10	r10d	r10w		r10b
r11	r11d	r11w		r11b
r12	r12d	r12w		r12b
r13	r13d	r13w		r13b
r14	r14d	r14w		r14b
r15	r15d	r15w		r15b

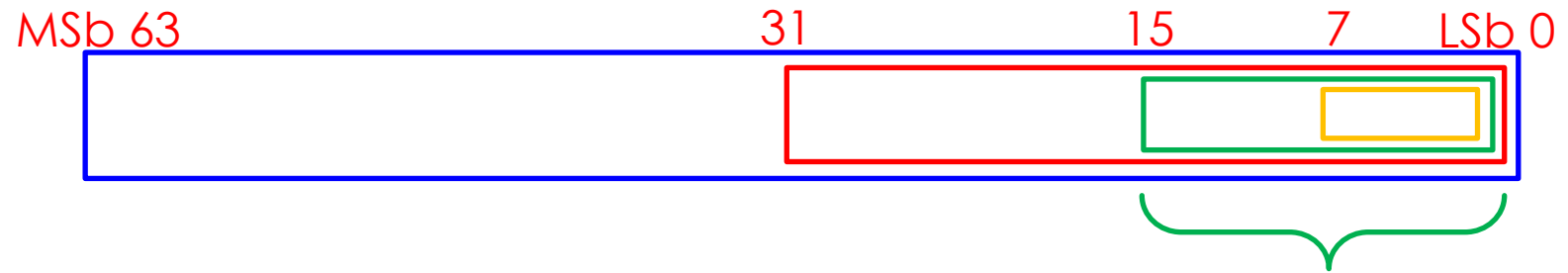
- Storage locations in CPU
-> fastest storage
- 16 registers are used explicitly – must name them in assembly code
- Some registers are used implicitly
 - e.g., PC, FLAGS
- Each register is 64 bits in size, but we can refer to its:
 - first byte LSB (8 bits),
 - first 2 bytes (16 bits),
 - first 4 bytes (32 bits),
 - or to all of its 8 bytes (64 bits)

About these integer registers!



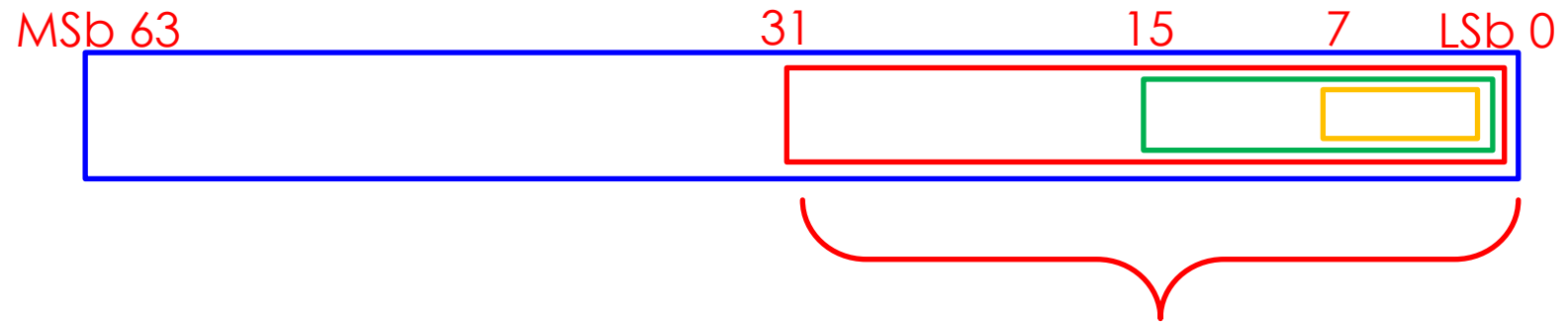
If I want 8 bits worth of data, then I can use register names such as `%a1` or `%di1` or `%r12b`

About these integer registers!



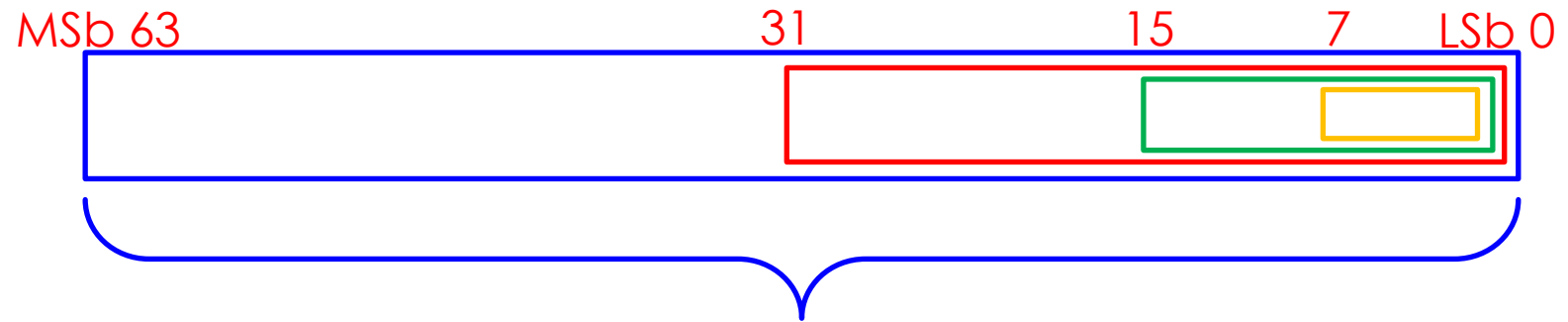
If I want **16** bits worth of data, then I can use register names such as **%ax** or **%di** or **%r12w**

About these integer registers!



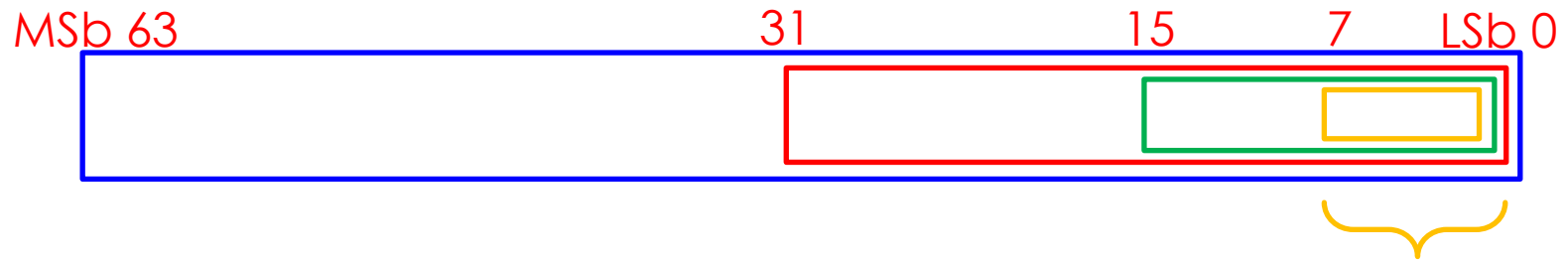
If I want **32** bits worth of data, then I can use register names such as **%eax** or **%edi** or **%r12d**

About these integer registers!



If I want **64** bits worth of data, then I can use register names such as **%rax** or **%rdi** or **%r12**

About these integer registers!



If I want **8** bits worth of data, then I can use register names such as **%a1** or **%di1** or **%r12b**

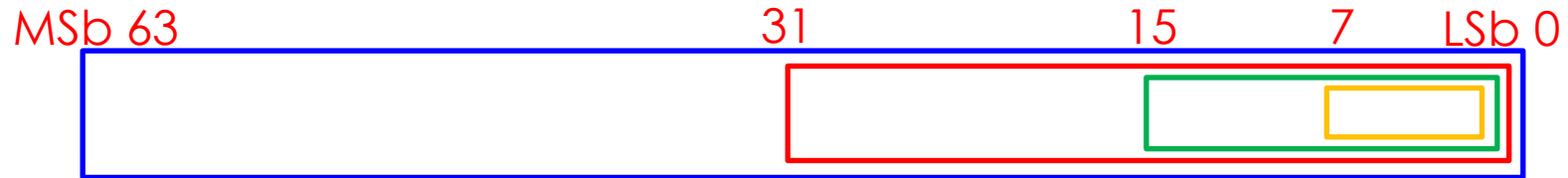
If I want **16** bits worth of data, then I can use register names such as **%ax** or **%di** or **%r12w**

If I want **32** bits worth of data, then I can use register names such as **%eax** or **%edi** or **%r12d**

If I want **64** bits worth of data, then I can use register names such as **%rax** or **%rdi** or **%r12**

Remember that for all 16 registers ...

Let's use the register associated with the names `%rax`, `%eax`, `%ax` and `%al` as an example:



- `%rax`, `%eax`, `%ax` and `%al` all refer to the **same register**
- However...
 - Each refer to a different section of this register
 - `%rax` refers to all 64 bits of this register
 - `%eax` refers to only 32 bits of this register
 - the LS 32 bits of it -> bit 0 to bit 31
 - `%ax` refers to only 16 bits of this register
 - the LS 16 bits of it -> bit 0 to bit 15
 - `%al` refers to only 8 bits of this register
 - the LS 8 bits of it -> bit 0 to bit 7

Summary

- Review: von Neumann architecture
 - Data and code are both stored in memory during program execution
- 1. How our **C** program/code can be represented as a series of 0's and 1's (i.e., as machine code)
 - Compilation process (gcc):
 - C** program -> assembly code -> machine level code -> executable (file)
 - Compiler made of tools: 1) **C** preprocessor, 2) **C** compiler, 3) assembler, 4) linker
- 2. Then, how our **C** program/code, once it is represented as a series of 0's and 1's, ends up being stored in memory
 - When **C** program is executed (tool: loader) (e.g. from our demo: `./ss 5 6`)
loader: loads file **ss** from external storage into memory
- 3. Then, how our **C** program (once it is represented as a series of 0's and 1's and it is stored in memory) ends up being executed by the microprocessor (aka the CPU)
 - CPU executes **C** program by looping through the fetch-execute cycle

Summary

- As **x86-64** assembly s/w dev., we now get to see more of the microprocessor (CPU): PC, registers, condition codes
- **x86-64** assembly language – **Data**
 - 16 integer registers of 64 bits each: can manipulate either 1, 2, 4 or 8 bytes (memory address of 8 bytes as well) depending on the register name used
 - Floating point registers of 64 bits each: can manipulate either 4 or 8 bytes depending on the register name used
 - No compound data types such as arrays or structures

Next Lecture

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations